

Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет
информатики и радиоэлектроники»

Факультет компьютерных систем и сетей

Кафедра информатики

Дисциплина: Методы оптимизации и управления

ОТЧЁТ
к лабораторной работе №6
на тему

ТРАНСПОРТНАЯ ЗАДАЧА

Выполнил: студент гр.353504
Левшуков Д.А.
Проверил:
Горбуков Д.М.

Минск 2026

СОДЕРЖАНИЕ

1 Постановка задачи.....	3
2 Краткие теоретические сведения.....	4
3 Описание функций программы.....	6
4 Пример выполнения программы	9
Вывод.....	11
Список использованных источников	12
Приложение А (обязательное) Исходные коды программ	13

1 ПОСТАНОВКА ЗАДАЧИ

Целью данной лабораторной работы является программная реализация эффективного алгоритма решения транспортной задачи – классической задачи линейного программирования, заключающейся в нахождении оптимального плана перевозок однородного груза от нескольких поставщиков к нескольким потребителям с минимальной суммарной стоимостью перевозок. Исходные данные задаются в формате JSON и включают количество поставщиков m , количество потребителей n , вектор запасов поставщиков a , вектор потребностей потребителей b и матрицу стоимостей перевозок c размера $m \times n$. На основе этих данных требуется определить оптимальный план перевозок x , минимизирующий целевую функцию, а также вычислить множество базисных клеток, образующих опорный план.

Входной файл должен иметь строго определённую структуру: объект, содержащий поля « m » (количество поставщиков), « n » (количество потребителей), « a » (массив запасов поставщиков размерности m), « b » (массив потребностей потребителей размерности n) и « c » (матрица стоимостей перевозок размерности $m \times n$). Все массивы и матрицы представляются вложенными массивами чисел, размерности строго согласованы с полями « m » и « n ». Программа должна корректно обрабатывать числа как в целочисленном, так и в вещественном формате, включая отрицательные значения (однако в корректной постановке задачи стоимости перевозок неотрицательны). В процессе синтаксического разбора необходимо заполнить внутренние структуры данных (вектор a , вектор b , матрицу c) и проверить корректность входных данных: соответствие размеров, наличие всех обязательных полей, неотрицательность запасов и потребностей, выполнение условия баланса — равенства суммарных запасов и суммарных потребностей.

Выходные данные представляют собой JSON-файл с именем `output.json`, содержащий следующие поля: «`status`» (статус решения, например «`optimal`»), «`objective`» (значение целевой функции — минимальная суммарная стоимость перевозок), « x » (матрица оптимального плана перевозок размера $m \times n$) и «`basis`» (список базисных клеток в формате $[[i, j], \dots]$, где индексация начинается с 1). Формат вывода соответствует структуре входного файла и позволяет наглядно проверить правильность выполнения всех шагов алгоритма. Программа должна корректно обрабатывать транспортные задачи произвольных размеров (в пределах установленных ограничений на память), а в случае обнаружения ошибок во входных данных (например, нарушения условия баланса, отрицательных запасов или потребностей, некорректной размерности матриц) или при вычислениях выдавать диагностические сообщения в стандартный поток ошибок. Реализация выполняется на языке Python.

2 КРАТКИЕ ТЕОРИТИЧЕСКИЕ СВЕДЕНИЯ

Транспортная задача является частным случаем задачи линейного программирования и формулируется следующим образом. Имеется m пунктов производства (поставщиков), в каждом из которых находится a_i единиц однородного груза, и n пунктов потребления (потребителей), каждому из которых требуется b_j единиц груза. Известна матрица стоимостей перевозок c_{ij} — стоимость перевозки одной единицы груза от i -го поставщика к j -му потребителю. Требуется найти такой план перевозок x_{ij} (количество груза, перевозимого от i -го поставщика к j -му потребителю), чтобы суммарная стоимость всех перевозок была минимальной, все запасы поставщиков были вывезены, а все потребности потребителей удовлетворены. Математическая модель транспортной задачи имеет вид: минимизировать целевую функцию $Z = \sum_{i=1}^m \sum_{j=1}^n c_{ij} x_{ij}$ при ограничениях $\sum_{j=1}^n x_{ij} = a_i$ для всех i (запасы поставщиков), $\sum_{i=1}^m x_{ij} = b_j$ для всех j (потребности потребителей) и $x_{ij} \geq 0$ для всех i, j .

Необходимым и достаточным условием существования допустимого плана транспортной задачи является выполнение условия баланса: суммарные запасы поставщиков должны равняться суммарным потребностям потребителей, то есть $\sum_{i=1}^m a_i = \sum_{j=1}^n b_j$, а также неотрицательность всех a_i и b_j . При выполнении этих условий транспортная задача всегда имеет хотя бы одно допустимое решение, например $x_{ij} = a_i b_j / S$, где S — суммарный объём запасов (или потребностей). Матрица коэффициентов системы ограничений транспортной задачи имеет размерность $(m+n) \times (m \cdot n)$ и состоит из нулей и единиц, причём каждый столбец содержит ровно две единицы: одну в i -й строке (соответствующей поставщику) и одну в $(m+j)$ -й строке (соответствующей потребителю). Ранг этой матрицы равен $m+n-1$, что обусловлено линейной зависимостью строк: сумма первых m строк равна сумме последних n строк, поэтому одно из ограничений является избыточным и может быть исключено.

Базисным допустимым планом транспортной задачи называется допустимый план, у которого количество положительных перевозок не превышает $m+n-1$, а набор клеток, соответствующих положительным перевозкам, не содержит циклов. Циклом в транспортной таблице называется последовательность клеток, в которой соседние клетки находятся в одной строке или в одном столбце, первая и последняя клетки также находятся в одной строке или в одном столбце, и никакие три клетки цикла не лежат в одной строке или в одном столбце. Условие отсутствия циклов в множестве базисных клеток эквивалентно линейной независимости соответствующих столбцов матрицы ограничений. Построение начального базисного допустимого плана может быть выполнено различными методами, среди которых наиболее простым является метод северо-западного угла, последовательно заполняющий клетки таблицы, начиная с левой верхней клетки, и распределяющий груз максимально возможным количеством, после

чего переходящий к соседней клетке вправо или вниз в зависимости от того, какой из запасов или потребностей оказался исчерпанным.

Потенциалами транспортной задачи называются переменные u_i (для поставщиков) и v_j (для потребителей), связанные с базисными клетками соотношениями $u_i + v_j = c_{ij}$. Поскольку количество базисных клеток равно $m+n-1$, а количество неизвестных потенциалов равно $m+n$, система имеет бесконечно много решений, и для определённости один из потенциалов фиксируется произвольным значением, обычно $u_1 = 0$. Для всех небазисных клеток вычисляются оценки $\Delta_{ij} = c_{ij} - u_i - v_j$. Критерий оптимальности базисного допустимого плана: если для всех небазисных клеток $\Delta_{ij} \leq 0$ (для задачи на минимум; для задачи на максимум — $\Delta_{ij} \geq 0$), то текущий план является оптимальным. Если существует клетка с положительной оценкой $\Delta_{ij} > 0$ (для задачи на минимум), то введение этой клетки в базис позволяет улучшить план, уменьшив целевую функцию.

Улучшение плана производится следующим образом. Для выбранной небазисной клетки (i, j) с $\Delta_{ij} > 0$ строится единственный цикл, в котором эта клетка является угловой, а все остальные угловые клетки являются базисными. Вершинам цикла поочерёдно присваиваются знаки «плюс» и «минус», начиная с «плюса» в клетке (i, j) . Затем определяется величина перераспределения $\theta = \min\{x_{kl} : (k, l) \text{ — клетка цикла со знаком «минус»}\}$. Далее ко всем клеткам цикла со знаком «плюс» прибавляется θ , из всех клеток со знаком «минус» вычитается θ . При этом клетка, на которой достигался минимум, обнуляется и выводится из базиса, а клетка (i, j) вводится в базис. Полученный новый план снова проверяется на оптимальность, и процесс повторяется до тех пор, пока все оценки Δ_{ij} не станут неположительными. Поскольку число базисных планов конечно, алгоритм гарантированно завершается за конечное число итераций (при условии предотвращения закливания, например с помощью правила выбора вводимой в базис клетки с наименьшим индексом).

Особое внимание следует уделить случаю транспортной задачи с неправильным балансом, когда суммарные запасы не равны суммарным потребностям. В такой ситуации задача либо не имеет решения (если запасов меньше, чем потребностей, или наоборот), либо требует введения фиктивного поставщика или фиктивного потребителя с нулевыми стоимостями перевозок для балансировки. В данной лабораторной работе предполагается, что входные данные корректны и условие баланса выполняется. После нахождения оптимального плана перевозок вычисляется минимальная суммарная стоимость перевозок $Z_{\min} = \sum_{i=1}^m \sum_{j=1}^n c_{ij} x_{ij}^*$, где x_{ij}^* — значения переменных в оптимальном решении. Полученное решение является оптимальным и может быть единственным или множественным (в случае наличия альтернативных оптимумов, что происходит при наличии нулевых оценок для некоторых небазисных клеток в оптимальном плане) [1].

3 ОПИСАНИЕ ФУНКЦИЙ ПРОГРАММЫ

Разработанная программа представляет собой реализацию транспортного симплекс-метода для решения транспортной задачи, написанную на языке Python. Код можно разделить на несколько функциональных частей, каждая из которых отвечает за определённый этап обработки данных и вычислительный процесс.

Первая часть – модуль чтения входных данных. Функция `read_input` открывает json-файл, переданный аргументом командной строки, и десериализует его содержимое. Проверяется наличие обязательных полей: `m` – количество поставщиков, `n` – количество потребителей, `a` – вектор запасов, `b` – вектор потребностей, `c` – матрица стоимостей. Все значения извлекаются в переменные соответствующего типа. В случае ошибок, таких как отсутствие файла, неверный формат json или несоответствие размерностей, программа выводит диагностическое сообщение в стандартный поток ошибок и завершается. После успешного чтения данные передаются в вычислительную часть.

Вторая часть – построение начального допустимого плана методом северо-западного угла. Функция `northwest_corner` создаёт копии векторов запасов и потребностей и инициализирует нулевую матрицу перевозок `x`. Указатели строки `i` и столбца `j` устанавливаются на левую верхнюю клетку таблицы. В цикле, пока существуют неисчерпанные поставщики и потребители, в текущую клетку помещается минимальное из оставшихся значений `a_i` и `b_j`. Затем запасы и потребности уменьшаются на эту величину, а клетка добавляется в множество базисных `B`. Если запас поставщика стал нулевым, указатель `i` увеличивается; если потребность потребителя стала нулевой – увеличивается `j`. Если после завершения цикла количество базисных клеток оказывается меньше $m+n-1$, вызывается вспомогательная функция `add_artificial_basis`, которая добавляет нулевые перевозки в клетки, соединяющие компоненты связности графа базиса с использованием системы непересекающихся множеств. Это гарантирует, что начальный базисный план является связным и имеет нужную размерность.

Третья часть – вычисление потенциалов и проверка оптимальности. Функция `compute_potentials` по базисным клеткам строит систему уравнений $u_i + v_j = c_{ij}$. Для решения строится двудольный граф, где рёбрами являются базисные клетки, и с помощью поиска в ширину от u_1 равного нулю вычисляются все u_i и v_j . Если в графе остаются несколько компонент связности, оставшиеся потенциалы устанавливаются в ноль. Далее для всех небазисных клеток вычисляются оценки δ_{ij} как c_{ij} минус u_i минус v_j . Критерий оптимальности для задачи минимизации: если все δ_{ij} больше или равны нулю, текущий план является оптимальным. При наличии отрицательной оценки выбирается лексикографически минимальная клетка с наименьшей (наиболее отрицательной) оценкой для ввода в базис.

Четвёртая часть – построение цикла пересчёта и улучшение плана. Функция `find_cycle` для множества базисных клеток и добавляемой

небазисной клетки строит единственный простой цикл. Сначала формируется список всех клеток объединения базиса и вводимой клетки, затем многократно удаляются строки и столбцы, содержащие не более одной клетки из этого списка. Оставшиеся клетки являются угловыми вершинами цикла. Далее порядок обхода восстанавливается чередованием движения по строке и по столбцу, начиная с вводимой клетки. Возвращается список клеток цикла в порядке обхода. Затем определяется величина перераспределения θ как минимальное значение перевозок среди клеток цикла, стоящих на нечётных позициях (знак «минус»). При нескольких минимумах выбирается лексикографически минимальная клетка для вывода из базиса. Выполняется пересчёт плана: ко всем клеткам цикла на чётных позициях прибавляется θ , из клеток на нечётных позициях вычитается θ . Клетка с минимальным значением обнуляется и исключается из базиса, а вводимая клетка добавляется. Полученный план становится новым базисным допустимым планом.

Пятая часть – вспомогательные функции и вывод результатов. Функция `print_plan` выводит на экран таблицу перевозок с разделительными линиями, справа от каждой строки – запас соответствующего поставщика, снизу от каждого столбца – потребность потребителя. Функция `transport_solver` организует итерационный процесс: сначала строится начальный план, затем в цикле выполняются улучшения до выполнения критерия оптимальности, после чего возвращается оптимальная матрица перевозок и минимальная стоимость. Главная функция `main` читает имя входного файла из аргументов командной строки, вызывает `read_input`, затем `transport_solver` и выводит итоговый план и стоимость. При установке флага `debug` в значение `true` программа печатает подробную информацию о каждой итерации: потенциалы, оценки, выбранные клетки, циклы, значение θ . Все промежуточные и итоговые данные обрабатываются с плавающей точкой, что позволяет решать задачи с вещественными запасами, потребностями и стоимостями. Управление памятью в Python осуществляется автоматическим сборщиком мусора, дополнительного освобождения ресурсов не требуется.

4 ПРИМЕР ВЫПОЛНЕНИЯ ПРОГРАММЫ

Для проверки правильности работы разработанной программы был подготовлен тестовый пример input.json представленный в приложении А. На рисунке 4.1 представлен результат работы программы на данном тестовом примере.

```
PS C:\Prog\BSUIR-Labs\Optimization and control methods\Lab 6> python.exe .\main.py .\input.json
Цикл (в порядке обхода): [(0, 2), (0, 0), (2, 0), (2, 2)]
theta = 100, выводимая клетка: (0, 0)
Обновлённый план:
  0.00    0.00   100.00   |   100.00
 100.00   200.00    0.00   |   300.00
 200.00    0.00   100.00   |   300.00
-----
 300.00   200.00   200.00
Новый базис: [(0, 2), (1, 0), (1, 1), (2, 0), (2, 2)]

--- Итерация 3 ---
Потенциалы u: [0, 3, 4]
Потенциалы v: [5, 1, 1]
  delta(0,0) = 8 - 0.00 - 5.00 = 3.00
  delta(0,1) = 4 - 0.00 - 1.00 = 3.00
  delta(1,2) = 3 - 3.00 - 1.00 = -1.00
  delta(2,1) = 7 - 4.00 - 1.00 = 2.00
Выбрана вводимая клетка (1, 2) с оценкой -1.00
Цикл (в порядке обхода): [(1, 2), (1, 0), (2, 0), (2, 2)]
theta = 100, выводимая клетка: (1, 0)
Обновлённый план:
  0.00    0.00   100.00   |   100.00
  0.00   200.00   100.00   |   300.00
 300.00    0.00    0.00   |   300.00
-----
 300.00   200.00   200.00
Новый базис: [(0, 2), (1, 1), (1, 2), (2, 0), (2, 2)]

--- Итерация 4 ---
Потенциалы u: [0, 2, 4]
Потенциалы v: [5, 2, 1]
  delta(0,0) = 8 - 0.00 - 5.00 = 3.00
  delta(0,1) = 4 - 0.00 - 2.00 = 2.00
  delta(1,0) = 8 - 2.00 - 5.00 = 1.00
  delta(2,1) = 7 - 4.00 - 2.00 = 1.00
Все оценки неотрицательны -> оптимальный план достигнут.

=== ОКОНЧАТЕЛЬНЫЙ ОПТИМАЛЬНЫЙ ПЛАН ===
  0.00    0.00   100.00   |   100.00
  0.00   200.00   100.00   |   300.00
 300.00    0.00    0.00   |   300.00
-----
 300.00   200.00   200.00
Общая стоимость: 3900.00
PS C:\Prog\BSUIR-Labs\Optimization and control methods\Lab 6> █
```

Рисунок 4.1 – Результат работы программы

ВЫВОД

В ходе выполнения лабораторной работы была разработана программа на языке Python, реализующая транспортный симплекс-метод для решения транспортной задачи – классической задачи линейного программирования с матрицей ограничений специальной структуры. Программа читает входной JSON-файл, извлекает количество поставщиков, количество потребителей, вектор запасов, вектор потребностей и матрицу стоимостей перевозок, а также проверяет корректность структуры данных. При любых отклонениях от ожидаемого формата или нарушении условий баланса и неотрицательности выдаются диагностические сообщения.

Для сбалансированных задач программа строит начальный опорный план методом северо-западного угла, что гарантирует получение базисного допустимого плана с числом базисных клеток, равным $m + n - 1$, включая искусственно добавленные нулевые перевозки для связности графа базиса. Затем выполняется оптимизация плана классическим методом потенциалов: вычисляются потенциалы строк и столбцов, для всех небазисных клеток определяются оценки, и при наличии отрицательной оценки строится цикл пересчёта, после чего производится перераспределение перевозок с выбором максимально возможного шага. Процесс повторяется до выполнения критерия оптимальности (все оценки неотрицательны). Лексикографический выбор вводимой и выводимой клеток обеспечивает за конечное число итераций.

Проверка корректности полученного решения осуществляется визуально через вывод на экран в табличном формате, а также автоматически через сохранение результатов в выходной файл `output.json`. Выходной JSON-файл содержит статус решения, значение целевой функции, матрицу оптимального плана перевозок и множество базисных клеток, что позволяет проводить дальнейший анализ и верификацию. Программа принимает имя входного файла в качестве аргумента командной строки, что делает её удобной для пакетной обработки нескольких тестовых примеров.

Разработанная программа полностью соответствует требованиям лабораторной работы: она выполняет чтение и проверку входных данных, реализует эффективный алгоритм решения транспортной задачи (метод северо-западного угла и метод потенциалов), корректно обрабатывает совместные задачи, а также сохраняет подробные результаты. Использование высокоуровневого языка Python обеспечивает наглядность и компактность кода, а автоматическое управление памятью избавляет от необходимости ручного освобождения ресурсов. Таким образом, поставленная цель лабораторной работы достигнута в полном объёме.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

[1] Костюкова, О.И. Методы оптимизации и управления / О.И. Костюкова, О.И. Дугинов

ПРИЛОЖЕНИЕ А
(обязательное)
Исходные коды программ

```
#input.json
{
    "m": 3,
    "n": 3,
    "a": [100, 300, 300],
    "b": [300, 200, 200],
    "c": [
        [8, 4, 1],
        [8, 4, 3],
        [9, 7, 5]
    ]
}

#main.py
import json
import sys
from collections import deque, defaultdict

def read_input(filename):
    with open(filename, 'r') as f:
        data = json.load(f)
    return data['m'], data['n'], data['a'], data['b'], data['c']

def northwest_corner(m, n, a, b):
    a_copy = a[:]
    b_copy = b[:]
    x = [[0]*n for _ in range(m)]
    B = set()
    i, j = 0, 0
    while i < m and j < n:
        val = min(a_copy[i], b_copy[j])
        x[i][j] = val
        a_copy[i] -= val
        b_copy[j] -= val
        B.add((i, j))
        if a_copy[i] == 0:
            i += 1
        if b_copy[j] == 0:
            j += 1
    if len(B) < m + n - 1:
        B = add_artificial_basis(m, n, B, x)
    return x, B

def add_artificial_basis(m, n, B, x):
    parent = list(range(m + n))
    rank = [0]*(m + n)
    def find(v):
        while parent[v] != v:
            parent[v] = parent[parent[v]]
    for (i, j) in B:
        find(i)
        find(j)
        parent[i] = parent[j]
        rank[i] += 1
        rank[j] += 1
    for i in range(m):
        find(i)
    for j in range(n):
        find(j)
    for i in range(m):
        for j in range(n):
            if (i, j) not in B:
                find(i)
                find(j)
                parent[i] = parent[j]
                rank[i] += 1
                rank[j] += 1
    return parent, rank
```

```

        v = parent[v]
    return v
def union(v1, v2):
    r1, r2 = find(v1), find(v2)
    if r1 == r2:
        return False
    if rank[r1] < rank[r2]:
        parent[r1] = r2
    elif rank[r1] > rank[r2]:
        parent[r2] = r1
    else:
        parent[r2] = r1
        rank[r1] += 1
    return True
for i, j in B:
    union(i, m + j)
for i in range(m):
    for j in range(n):
        if (i, j) not in B and find(i) != find(m + j):
            B.add((i, j))
            x[i][j] = 0
            union(i, m + j)
            if len(B) == m + n - 1:
                break
    if len(B) == m + n - 1:
        break
return B

def compute_potentials(m, n, B, c):
    u = [None]*m
    v = [None]*n
    adj = [[] for _ in range(m + n)]
    for i, j in B:
        adj[i].append((m + j, c[i][j]))
        adj[m + j].append((i, c[i][j]))
    u[0] = 0
    q = deque([0])
    while q:
        node = q.popleft()
        for nb, cost in adj[node]:
            if node < m: # строка -> столбец
                if v[nb - m] is None:
                    v[nb - m] = cost - u[node]
                    q.append(nb)
            else: # столбец -> строка
                if u[nb] is None:
                    u[nb] = cost - v[node - m]
                    q.append(nb)
    for i in range(m):
        if u[i] is None:
            u[i] = 0
    for j in range(n):
        if v[j] is None:

```

```

        v[j] = 0
    return u, v

def find_cycle(B, enter_cell, m, n):
    cells = list(B) + [enter_cell]
    row_cnt = [0]*m
    col_cnt = [0]*n
    for i, j in cells:
        row_cnt[i] += 1
        col_cnt[j] += 1
    removed_rows = [False]*m
    removed_cols = [False]*n
    changed = True
    while changed:
        changed = False
        for i in range(m):
            if not removed_rows[i] and row_cnt[i] <= 1:
                to_remove = [(r, c) for (r, c) in cells if r ==
i]
                    for rc in to_remove:
                        cells.remove(rc)
                        row_cnt[rc[0]] -= 1
                        col_cnt[rc[1]] -= 1
                        removed_rows[i] = True
                        changed = True
        for j in range(n):
            if not removed_cols[j] and col_cnt[j] <= 1:
                to_remove = [(r, c) for (r, c) in cells if c ==
j]
                    for rc in to_remove:
                        cells.remove(rc)
                        row_cnt[rc[0]] -= 1
                        col_cnt[rc[1]] -= 1
                        removed_cols[j] = True
                        changed = True
    if not cells:
        return []
    rows = defaultdict(list)
    cols = defaultdict(list)
    for i, j in cells:
        rows[i].append(j)
        cols[j].append(i)
    start = enter_cell
    cycle = [start]
    curr = start
    step = 0
    while True:
        i_curr, j_curr = curr
        if step % 2 == 0:
            for j in rows[i_curr]:
                if j != j_curr:
                    nxt = (i_curr, j)
                    break

```

```

        else:
            for i in cols[j_curr]:
                if i != i_curr:
                    nxt = (i, j_curr)
                    break
            if nxt == start:
                break
            cycle.append(nxt)
            curr = nxt
            step += 1
    return cycle

def transport_solver(m, n, a, b, c, debug=False):
    if sum(a) != sum(b):
        raise ValueError("Задача не сбалансирована")
    x, B = northwest_corner(m, n, a, b)
    if debug:
        print("\n=== НАЧАЛЬНЫЙ ПЛАН (северо-западный угол) ===")
        print_plan(x, a, b)
        print(f"Базис: {sorted(B)}")
    iteration = 0
    while True:
        iteration += 1
        u, v = compute_potentials(m, n, B, c)
        if debug:
            print(f"\n--- Итерация {iteration} ---")
            print("Потенциалы u:", [round(val, 2) for val in u])
            print("Потенциалы v:", [round(val, 2) for val in v])
        min_delta = float('inf')
        enter_cell = None
        for i in range(m):
            for j in range(n):
                if (i, j) not in B:
                    delta = c[i][j] - u[i] - v[j]
                    if debug:
                        print(f"delta({i},{j}) = {c[i][j]} - {u[i]:.2f} - {v[j]:.2f} = {delta:.2f}")
                    if delta < 0:
                        if delta < min_delta or (delta == min_delta and (i, j) < enter_cell):
                            min_delta = delta
                            enter_cell = (i, j)
        if enter_cell is None:
            if debug:
                print("Все оценки неотрицательны -> оптимальный план достигнут.")
            break
        if debug:
            print(f"Выбрана вводимая клетка {enter_cell} с оценкой {min_delta:.2f}")
        cycle = find_cycle(B, enter_cell, m, n)
        if debug:
            print("Цикл (в порядке обхода):", cycle)

```

```

        theta = float('inf')
        exit_cell = None
        for idx, (i, j) in enumerate(cycle):
            if idx % 2 == 1:
                if x[i][j] < theta or (x[i][j] == theta and (i,
j) < exit_cell):
                    theta = x[i][j]
                    exit_cell = (i, j)
            if debug:
                print(f"theta = {theta}, выводимая клетка:
{exit_cell}")
        for idx, (i, j) in enumerate(cycle):
            if idx % 2 == 0:
                x[i][j] += theta
            else:
                x[i][j] -= theta
        B.remove(exit_cell)
        B.add(enter_cell)
        if debug:
            print("Обновлённый план:")
            print_plan(x, a, b)
            print(f"Новый базис: {sorted(B)}")
        total_cost = sum(c[i][j] * x[i][j] for i in range(m) for j
in range(n))
        return x, total_cost

def print_plan(x, a, b):
    m, n = len(x), len(x[0])
    for i in range(m):
        row_str = ' '.join(f"{x[i][j]:8.2f}" for j in range(n))
        print(f"{row_str} | {a[i]:8.2f}")
    print('-' * (12 * n + 10))
    for j in range(n):
        print(f"{b[j]:8.2f} ", end='')
    print()

def main():
    if len(sys.argv) != 2:
        print("Usage: python main.py <input.json>")
        sys.exit(1)
    filename = sys.argv[1]
    m, n, a, b, c = read_input(filename)
    x, total_cost = transport_solver(m, n, a, b, c, debug=True)
    print("\n=== ОКОНЧАТЕЛЬНЫЙ ОПТИМАЛЬНЫЙ ПЛАН ===")
    print_plan(x, a, b)
    print(f"Общая стоимость: {total_cost:.2f}")

if __name__ == "__main__":
    main()

```